

Patching-based Interpretability Graphs and Kernel Methods

Mechanistic Interpretability $\rightarrow$ Graph Datasets $\rightarrow$ Similarity Geometry

MI Working Group

Uvigo

Roadmap

- 1 Problem Setting: Mechanistic Interpretability
- 2 Patching: From Interventions to Effect Maps
- 3 Theoretical Formulation
- 4 Algorithms: From Patching to Kernels
- 5 Execution Plan: Step-by-step Tasks & Expected Outputs
- 6 Evaluation Checklist
- 7 Conclusion

Motivation: Mechanistic Interpretability as a *causal structure discovery*

Problem statement (why patching exists)

Transformer models implement complex computations distributed across **layers, tokens, heads, and MLPs**. Mechanistic interpretability asks:

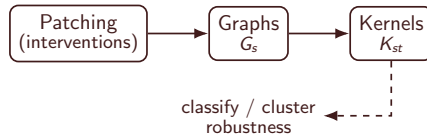
- **Which internal components causally influence** a target behavior (e.g., next-token logit)?
- **How do components compose into circuits** that implement the computation?
- **How stable are these circuits** across prompts, corruptions, and noise?

Key obstacle

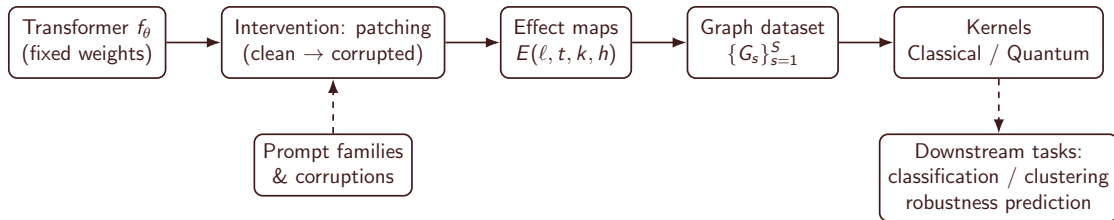
Purely observational probes (activations, correlations) confound *co-activation* with *causal influence*. We need **interventions** to map *cause* → *effect* inside the network.

Core idea of this project

Use patching to produce a **large interventional dataset**, summarize it as a **graph dataset**, then use kernels (classical / quantum) to learn an **effective similarity geometry** over circuits.



Conceptual pipeline (from interventions to a learnable dataset)



What becomes the dataset?

Each slice s (task family, corruption type, prompt family) yields one graph G_s . Learning happens *across graphs*, not just within a single prompt.

Patching: controlled interventions inside the forward pass

Clean vs corrupted runs

For each example i , we build a pair:

$$(x_i^{\text{cln}}, x_i^{\text{crp}}),$$

where x_i^{crp} is designed to break the target behavior while preserving structure.

Effect size (scalar observable)

Let $O(\cdot)$ be a scalar observable (logit, log-prob, loss). Define the patch effect:

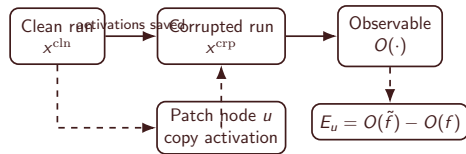
$$E_u(x^{\text{cln}}, x^{\text{crp}}) = O(\tilde{f}_\theta(x^{\text{crp}}; u)) - O(f_\theta(x^{\text{crp}})).$$

Large E_u means that intervening at u causally restores the target behavior.

Activation patching (local intervention)

Choose an internal node u (layer/token/component/head). Run the corrupted forward pass, but **replace** the activation at u with the corresponding clean activation.

Conceptual diagram



Formal setup: model, observable, and component activations

Model and observable

Let f_θ be a transformer. For a prompt $x = (x_1, \dots, x_T)$, denote logits by $\ell(x) \in \mathbb{R}^{|V|}$. Choose an observable:

$$O(x) = \ell_{y^*}(x) \quad \text{or} \quad O(x) = -\log p_\theta(y^* | x),$$

where y^* is the target token.

Component types and indices

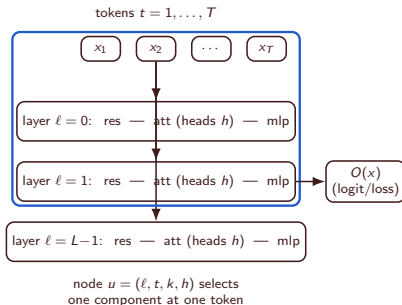
Let $\ell \in \{0, \dots, L-1\}$ (layer), $t \in \{1, \dots, T\}$ (token index),

$$\mathcal{K} := \{\text{res}, \text{att}, \text{mlp}\}, \quad h \in \{1, \dots, H\} \text{ (heads, only for att)}.$$

Example activations:

$$a_{\ell,t}^{\text{res}}(x) \in \mathbb{R}^d, \quad a_{\ell,t,h}^{\text{att}}(x) \in \mathbb{R}^d, \quad a_{\ell,t}^{\text{mlp}}(x) \in \mathbb{R}^d.$$

Conceptual view (what is indexed by (ℓ, t, k, h) ?)



Nodes & graphs: mathematical definition (what the dataset actually is)

Node set (mechanistic granularity)

Fix (L, T, H) . Define the node set as a disjoint union:

$$\mathcal{V} = \mathcal{V}_{\text{res}} \dot{\cup} \mathcal{V}_{\text{mlp}} \dot{\cup} \mathcal{V}_{\text{att}},$$

where

$$\mathcal{V}_{\text{res}} = \{(\ell, t, \text{res}) : 0 \leq \ell < L, 1 \leq t \leq T\}, \quad \mathcal{V}_{\text{mlp}} = \{(\ell, t, \text{mlp}) : 0 \leq \ell < L, 1 \leq t \leq T\},$$

$$\mathcal{V}_{\text{att}} = \{(\ell, t, h, \text{att}) : 0 \leq \ell < L, 1 \leq t \leq T, 1 \leq h \leq H\}.$$

A generic node is $u \in \mathcal{V}$.

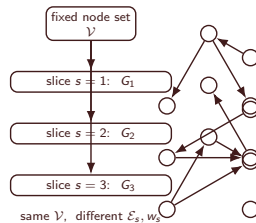
Graph per slice

For each slice s (task family / corruption type / prompt family), build a directed weighted graph:

$$G_s = (\mathcal{V}, \mathcal{E}_s, w_s).$$

Weights $w_s(u \rightarrow v)$ are computed from patch-effect profiles across examples in slice s .

Conceptual view (fixed nodes, varying edges by slice)



From patch effects to graphs: two principled constructions

(A) Influence-to-output graph (direct causal score)

Define node score as the expected causal restoration:

$$\text{score}_s(u) = \mathbb{E}_{i \in \mathcal{I}_s} [E_u^{(i)}],$$

and define directed edges by conditional/paired interventions (optional extension):

$$w_s(u \rightarrow v) \approx \mathbb{E}[E_u \text{ then } v - E_u],$$

capturing how v mediates the effect of u .

(B) Co-influence graph (robust, cheap, dataset-friendly)

Given patch-effect vectors $\{E_u^{(i)}\}_{i \in \mathcal{I}_s}$, define

$$w_s(u \rightarrow v) = \text{corr}(\{E_u^{(i)}\}_{i \in \mathcal{I}_s}, \{E_v^{(i)}\}_{i \in \mathcal{I}_s}), \quad \text{often use } |w_s(u \rightarrow v)|.$$

Interpretation: nodes whose interventions co-vary across prompts behave as a coherent circuit.

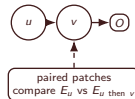
Direction constraint (recommended)

Allow edges only from earlier to later in a partial order $\prec$ (e.g. by layer, then token):

$$(u \rightarrow v) \in \mathcal{E}_s \Rightarrow u \prec v.$$

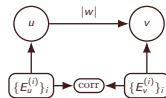
Conceptual view (edge definitions)

(A) Influence-to-output (mediation)
 $w(u \rightarrow v) \approx \mathbb{E}[E_u \text{ then } v - E_u]$



(often keep only $u \prec v$)

(B) Co-influence (correlation of profiles)
 $w(u \rightarrow v) = \text{corr}(E_u, E_v)$



(then sparsify: top-k edges per node)

Edge sparsification & comparability (make graphs scalable)

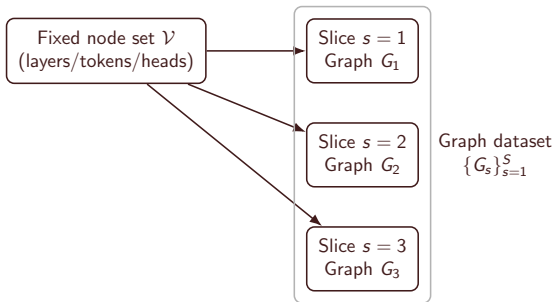
Top- k sparsification (critical)

For each $u \in \mathcal{V}$, keep only top- k outgoing edges by $|w_s(u \rightarrow v)|$:

$$\mathcal{E}_s = \bigcup_{u \in \mathcal{V}} \{(u \rightarrow v) : v \in \text{TopK}_k(\{|w_s(u \rightarrow \cdot)|\})\}.$$

This fixes graph size and stabilizes downstream kernels.

Conceptual view (same nodes, different slices)



Algorithm 1: Build patch-effect tensor and graph dataset

Algorithm 1: From patching to a dataset of sparse interpretability graphs.

Input : Transformer f_θ (fixed);

dataset $\{(x_i^{\text{cln}}, x_i^{\text{crp}}, y_i^*)\}_{i=1}^N$;

node set $\mathcal{V}$;

observable $O(\cdot)$;

slices $\{\mathcal{I}_s\}_{s=1}^S$;

direction constraint $\prec$ (optional);

top- k .

Output: Graph dataset $\{G_s = (\mathcal{V}, \mathcal{E}_s, w_s)\}_{s=1}^S$ and labels.

(A) Compute patch effects

for $i = 1, \dots, N$ do

$O_i^{\text{base}} \leftarrow O(f_\theta(x_i^{\text{crp}}))$

 foreach $u \in \mathcal{V}$ do

 Run patched forward $\tilde{f}_\theta(x_i^{\text{crp}}; u)$ by copying the activation specified by node u from x_i^{cln}

$E_u^{(i)} \leftarrow O(\tilde{f}_\theta(x_i^{\text{crp}}; u)) - O_i^{\text{base}}$

 end

end

(B) Build one graph per slice

foreach $s \in \{1, \dots, S\}$ do

 foreach $u \in \mathcal{V}$ do

 foreach $v \in \mathcal{V}$ do

 if $u \prec v$ or no constraint then

$w_s(u \rightarrow v) \leftarrow \text{corr}(\{E_u^{(i)}\}_{i \in \mathcal{I}_s}, \{E_v^{(i)}\}_{i \in \mathcal{I}_s})$

 end

 end

 Keep top- k edges $(u \rightarrow v)$ by $|w_s(u \rightarrow v)|$; add them to $\mathcal{E}_s$

 end

 Set $G_s \leftarrow (\mathcal{V}, \mathcal{E}_s, w_s)$

end

Intuition (what the code is doing)

- **Freeze the model.** We only run forwards; no training required.
- **Create paired inputs** $(x^{\text{cln}}, x^{\text{crp}})$ so corruption breaks the behavior.
- **For each node u :** copy the clean activation into the corrupted run (one intervention).
- **Measure causal effect:** does this patch restore the target logit/loss?

$$E_u^{(i)} = O(\tilde{f}; u) - O(f)$$

- **Aggregate by slice s :** build one graph per task/corruption family.
- **Edges = co-variation:** connect $u \rightarrow v$ if their effects co-vary across prompts (correlation $\{E_u^{(i)}\}$ and $\{E_v^{(i)}\}$).
- **Make it scalable:** keep only top- k outgoing edges per node (sparse, comparable graphs).

Outputs

- Patch-effect tensor $\{E_u^{(i)}\}$ (raw interventional dataset)
- Graph dataset $\{G_s\}$ (one graph per slice, same nodes)

Kernels: from graphs to similarity matrices (the “swap” point)

Step 1: graph embedding/features

Choose a graph feature map ϕ_{graph} :

$$x_s = \phi_{\text{graph}}(G_s) \in \mathbb{R}^d,$$

e.g. WL subtree features, spectral vectors, motif (graphlet) counts, or simple typed-degree histograms.

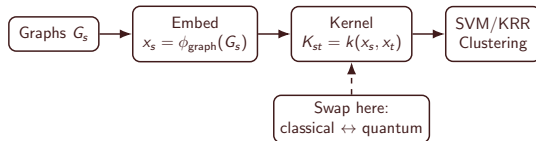
Step 2: similarity layer (classical vs quantum)

$$K_{st} = k(x_s, x_t).$$

- **Classical:** $k_{\text{lin}}(x, x') = x^\top x'$, $k_{\text{RBF}}(x, x') = \exp(-\gamma \|x - x'\|^2)$.
- **Quantum:** encode x with a feature map circuit $U_\phi(x)$, $|\phi(x)\rangle = U_\phi(x)|0\rangle$, and use the fidelity kernel

$$k_q(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2 = |\langle 0 | U_\phi(x)^\dagger U_\phi(x') | 0 \rangle|^2.$$

Conceptual diagram: only the kernel changes



Takeaway

- Graph construction and embeddings stay fixed.
- Only $k(\cdot, \cdot)$ changes $\Rightarrow$ geometry changes.
- Same downstream code (SVM/KRR/clustering).

Algorithm 2: Classical graph kernels (WL/spectral/graphlets)

Algorithm 2: Classical kernel learning on interpretability-graph embeddings.

Input : Graphs $\{G_s\}_{s=1}^S$; embedding ϕ_{graph} ; kernel type (linear/RBF); labels $\{y_s\}$ (optional).

Output: Kernel matrix $K \in \mathbb{R}^{S \times S}$ and downstream results.

for $s = 1, \dots, S$ **do**

$x_s \leftarrow \phi_{\text{graph}}(G_s) \in \mathbb{R}^d$

end

for $s = 1, \dots, S$ **do**

for $t = 1, \dots, S$ **do**

if *linear* **then**

$K_{st} \leftarrow x_s^\top x_t$

else if *RBF* **then**

$K_{st} \leftarrow \exp(-\gamma \|x_s - x_t\|^2)$

end

end

end

Train SVM/KRR on $(K, \{y_s\})$ or cluster using K

Intuition (what you get, step by step)

- **Compress each graph** G_s into a vector x_s using a standard graph feature map:
 - WL = “bag of subtree patterns” (fast, strong baseline)
 - Spectral = “shape of the graph” via eigenvalues/vectors
 - Graphlets = “motif counts” (small circuit motifs)
- **Turn vectors into similarities:** build $K_{st} = k(x_s, x_t)$.
- **Linear kernel:** similarity = dot product (works well with WL counts).
- **RBF kernel:** similarity decays with distance (nonlinear decision boundary).
- **Downstream learning uses only K :**
 - supervised: SVM / kernel ridge regression
 - unsupervised: kernel k-means / spectral clustering

Practical notes

- Normalize features (or kernel) to compare slices fairly.
- Tune γ (RBF) by validation or median heuristic.
- Cache x_s and K — they are reused by quantum swap later.

Algorithm 3: Quantum kernel swap (fidelity kernel on graph embeddings)

Algorithm 3: Quantum kernel as a drop-in similarity layer (no change to graph construction).

Input : Embeddings $\{x_s\}_{s=1}^S$; quantum feature map $U_\phi(\cdot)$; shots M ; downstream method (SVM/KRR/clustering).

Output: Quantum kernel matrix $K^{(q)}$ and downstream results + shot/noise ablations.

```
for  $s = 1, \dots, S$  do
  for  $t = 1, \dots, S$  do
    // Estimate  $k_q(x_s, x_t) = |\langle 0 | U_\phi(x_s)^\dagger U_\phi(x_t) | 0 \rangle|^2$ 
    Run overlap estimation (or swap-test style) circuit with  $M$ 
    shots
     $K_{st}^{(q)} \leftarrow \hat{k}_q(x_s, x_t)$ 
  end
end
```

Run SVM/KRR/clustering using $K^{(q)}$

Repeat for different circuit depths / encodings / M (shot noise)

Intuition (what changes vs classical)

- **Nothing upstream changes:** graphs $\rightarrow$ embeddings x_s stay identical.
- **Only the similarity changes:** replace k_{RBF} with a quantum fidelity kernel.
- **Each x becomes a quantum state:**

$$|\phi(x)\rangle = U_\phi(x) |0\rangle.$$

- **Similarity = state overlap:** large overlap $\Rightarrow$ “similar graphs” under this geometry.
- **Shots M control variance:** more shots $\Rightarrow$ lower Monte Carlo noise, higher cost.
- **Depth controls expressivity:** deeper circuits can yield richer kernels but may be noisier / harder to estimate.

Ablations to report (minimum)

- Accuracy/AUC vs shots M (shot-noise curve)
- Accuracy vs circuit depth / encoding choice
- Kernel spectrum / alignment vs classical baselines

Plan overview (deliverables by construction)

End-to-end deliverables

- A fixed local transformer + a controlled prompt/corruption generator.
- A patching engine that outputs effect tensors $E_u^{(i)}$.
- A graph dataset $\{G_s\}_{s=1}^S$ with standardized sparsity and attributes.
- Baselines: WL / spectral / graphlets $\Rightarrow$ classical kernels.
- Quantum-kernel swap on the same embeddings + shot/noise ablations.
- A minimal paper-style evaluation: classification, clustering, stability vs noise.

Guiding principle

Freeze complexity early: **one model, one task family, one patch type**, then expand only after the pipeline is stable.

Task 0: Choose a small local LLM (stable, many forward passes)

What will be done

Select a transformer that can run **many** forward passes + patch interventions per hour.

How to do it (practical choices)

- **Option A (simple):** nanoGPT-style small model trained locally on a small corpus.
- **Option B (open weights):** small GPT-like open model (keep it fixed; no heavy finetuning).
- Fix: context length T , layers L , heads H , and chosen internal components to patch.

What to expect / acceptance test

- You can run $\gtrsim 10^4$ forward passes in a reasonable session (scale depends on hardware).
- You can hook and replace activations at layer/token/head with reproducible results.

Task 0 (narrowed): Minimal local LLM setup + hook check

Goal (keep scope tight)

- Pick **one** small HF model that runs locally *fast*.
- Freeze weights; do **only forward passes + hooks**.
- Patch **one component first**: residual stream at (ℓ, t) .

Workflow (30-min acceptance test)

- ➊ Load model + tokenizer; fix T (truncate/pad).
- ➋ Run one clean + one corrupted prompt.
- ➌ Register a forward hook at layer ℓ to **capture** $a_{\ell,t}^{cs}$.
- ➍ Re-run corrupted prompt with a hook that **replaces** that activation by the clean one.
- ➎ Check: Δlogit on the target token changes in the expected direction.

Code skeleton (PyTorch + HF): capture + patch one residual stream

```
1 import torch
2 from transformers import AutoTokenizer, AutoModelForCausalLM
3
4 device = "cuda" if torch.cuda.is_available() else "cpu"
5 name = "gpt2" # keep it small + stable
6 tok = AutoTokenizer.from_pretrained(name)
7 model = AutoModelForCausalLM.from_pretrained(name).to(device)
8 model.eval()
9
10 def run(prompt):
11     x = tok(prompt, return_tensors="pt").to(device)
12     with torch.no_grad():
13         out = model(**x, output_hidden_states=True)
14     return out, x
15 # --- choose layer l and token index t ---
16 l, t = 6, -1 # last token position
17 clean_prompt = "Alice gave Bob a book. Bob thanked"
18 crpt_prompt = "Alice gave Bob a book. Alice thanked"
```

Code cont

```
1 out_cln, x_cln = run(clean_prompt)
2 out_crpt, x_crpt = run(crpt_prompt)
3
4 # hidden_states: tuple length L+1; take residual stream at layer l
5 a_clean = out_cln.hidden_states[l][0, t, :].detach().clone()
6
7 target_id = tok.encode(" Bob", add_special_tokens=False)[0]
8 base_logit = out_crpt.logits[0, t, target_id].item()
9
10 # Patch by modifying hidden state at layer l via a forward hook.
11 # (Hook target may differ by architecture; this is a template.)
12 handle = None
13 def patch_hook(module, inp, out):
14     # out shape: [B,T,d]
15     out = out.clone()
16     out[0, t, :] = a_clean
17     return out
18
19 # Example hook point: transformer block output (GPT-2)
20 handle = model.transformer.h[l].register_forward_hook(
21     lambda m, inp, out: patch_hook(m, inp, out)
22 )
23
24 with torch.no_grad():
25     out_p = model(**x_crpt, output_hidden_states=False)
26
27 handle.remove()
28 patched_logit = out_p.logits[0, t, target_id].item()
29 print("Dlogit =", patched_logit - base_logit)
```

Expected outcome

- You get a **nonzero** Δlogit for at least some (ℓ, t) .
- This validates: model runs locally + you can capture/replace activations.

Task 1: Define a controlled task family + corruption generator

What will be done

Choose tasks where causal structure is sharp and patching produces structured signals.

How to do it (examples)

- IOI-style templates (name swaps), key-value retrieval, bracket matching, simple arithmetic.
- Build **prompt families** (templates) and **corruptions** (swap/replace a key token).
- Define slice labels s (e.g. task type, corruption type, difficulty bin).

What to expect / acceptance test

- Clean prompts succeed; corrupted prompts fail (measurable via $O(x)$).
- The dataset can generate N paired examples $(x^{\text{cln}}, x^{\text{crp}})$ at scale.

What is IOI

IOI = Indirect Object Identification: prompts are written so the model should predict which name is the indirect object later in the sentence (“who got the thing?”), even when there’s a distractor name.

An IOI swap corruption means you take a clean IOI prompt and swap a key name (or swap the roles of the two names) so the model’s usual reasoning path breaks.

Task 1 (narrowed): One prompt template + one corruption + slice labels

Goal (keep scope tight)

- Implement **one** task family first: IOI-style name resolution.
- Implement **one** corruption: swap the indirect object.
- Output paired data: $(x^{\text{cln}}, x^{\text{crp}}, y^*)$ plus slice label s .

Workflow (acceptance test)

- 1 Sample names (A, B) and a verb/object pair.
- 2 Build a clean prompt where the correct next token is predictable (target y^*).
- 3 Build a corrupted prompt by swapping a key token (indirect object).
- 4 Evaluate $O(x)$ on both: **clean should score higher than corrupted**.
- 5 Store: `slice={task=IOI, corruption=swap}`.

What to expect

- A measurable gap: $\Delta O = O(x^{\text{cln}}) - O(x^{\text{crp}}) > 0$ on average.
- Stable generation of N pairs with fixed random seed (reproducible).

Scoring $O(x)$ (minimal choice)

- Use the logit/log-prob of token y^* at the last position:

$$O(x) = \ell_{y^*}(x) \quad \text{or} \quad O(x) = \log p_{\theta}(y^* | x).$$

- Acceptance: $\mathbb{E}[O(x^{\text{cln}}) - O(x^{\text{crp}})] > 0$.

Code skeleton: prompt template + corruption generator (IOI swap)

```
1 import random
2
3 NAMES = ["Alice", "Bob", "Carol", "Dave", "Eve", "Frank", "Grace", "Henry"]
4 OBJS = ["book", "letter", "cup", "ball", "key"]
5 VERBS = ["gave", "sent", "handed"]
6
7 def make_ioi_pair(rng: random.Random):
8     A, B = rng.sample(NAMES, 2)
9     obj = rng.choice(OBJS)
10    verb = rng.choice(VERBS)
11
12    # Clean: indirect object = B (so next token tends to be B)
13    x_cln = f"{A} {verb} {B} a {obj}. {B} thanked"
14    # Corrupt: swap indirect object to A (break the causal structure)
15    x_crp = f"{A} {verb} {B} a {obj}. {A} thanked"
16
17    # Target token: the "right" name after "thanked" depends on your scoring choice.
18    # Minimal: treat the target as the name that should follow (IOI-style).
19    y_star = B
20
21    slice_label = {"task": "IOI", "corruption": "swap"}
22    meta = {"A": A, "B": B, "obj": obj, "verb": verb}
23    return x_cln, x_crp, y_star, slice_label, meta
24
25 def generate_dataset(N, seed=0):
26     rng = random.Random(seed)
27     return [make_ioi_pair(rng) for _ in range(N)]
28
29 pairs = generate_dataset(5, seed=42)
30 for x_cln, x_crp, y, s, meta in pairs:
31     print(s, "y*=", y)
32     print(" clean:", x_cln)
33     print(" corrupt:", x_crp)
```

Task 2: Implement patching hooks (residual / attn / MLP)

What will be done

Implement a patch operator $\tilde{f}_\theta(x^{\text{crp}}; u)$ for nodes $u \in \mathcal{V}$ with a chosen granularity:

$$u = (\ell, t, \text{res}) \text{ or } (\ell, t, \text{mlp}) \text{ or } (\ell, t, h, \text{att}).$$

How to do it

- Cache clean activations for all patchable nodes.
- Re-run the corrupted forward pass, and at a single node u replace the activation tensor.
- Start with **residual stream patching** (most standard) before splitting into heads/MLPs.

What to expect / acceptance test

- Patching a “known important” node increases $O(\cdot)$ on corrupted prompts.
- Sanity check: patching random nodes has small average effect.

Task 2 (narrowed): Implement one patch hook (residual stream) + extendable API

Goal (keep scope tight)

- Implement patching for **one node type first**: $u = (\ell, t, \text{res})$.
- Provide a minimal API:

$$\text{patch_run}(\text{prompt_crp}, \text{cache_cln}, u) \mapsto O(\tilde{f}_\theta)$$

- Later, extend the same pattern to (ℓ, t, mlp) and (ℓ, t, h, att) .

Workflow (acceptance test)

- 1 Run clean prompt once and **cache** residual activations by layer.
- 2 Run corrupted prompt baseline and record O_{base} .
- 3 Re-run corrupted prompt with a hook that **replaces** residual at (ℓ, t) by clean.
- 4 Compute effect $E_u = O_{\text{patched}} - O_{\text{base}}$.
- 5 Sanity: random (ℓ, t) gives small $|E_u|$ on average.

Implementation note (robustness)

Hook the **block output** (residual stream after the transformer block) first. This is stable and easiest to reason about.

Code skeleton: cache clean residuals + patch one node (ℓ, t, res)

```
1 import torch
2 from transformers import AutoTokenizer, AutoModelForCausalLM
3
4 device = "cuda" if torch.cuda.is_available() else "cpu"
5 name = "gpt2"
6 tok = AutoTokenizer.from_pretrained(name)
7 model = AutoModelForCausalLM.from_pretrained(name).to(device).eval()
8
9 @torch.no_grad()
10 def score_logprob(prompt, target_str, t=-1):
11     x = tok(prompt, return_tensors="pt").to(device)
12     out = model(**x, output_hidden_states=False)
13     tid = tok.encode(target_str, add_special_tokens=False)[0]
14     return out.logits[0, t, tid].item(), x
```

Expected outcome

- For some (ℓ, t) , $E_u > 0$ (patch restores target logit).
- Random (ℓ, t) : E_u concentrates near 0 (sanity baseline).

Task 2 (narrowed): Implement one patch hook (residual stream) + extendable API

Code Cont

```
1 @torch.no_grad()
2 def cache_residuals(prompt, layers):
3     """
4     Returns: dict layer -> tensor [T,d] (residual stream snapshot)
5     For GPT-2: out.hidden_states[1] is residual stream at layer 1 (pre-block);
6     as a simple, consistent cache, store hidden_states[1] and patch at the same tap.
7     """
8     x = tok(prompt, return_tensors="pt").to(device)
9     out = model(**x, output_hidden_states=True)
10    cache = {}
11    for l in layers:
12        cache[l] = out.hidden_states[1][0].detach().clone() # [T,d]
13    return cache, x
14
15 def patch_hidden_states_layer(l, t, a_clean_Td):
16     """
17     Patch hook for the module that outputs a [B,T,d] tensor.
18     IMPORTANT: exact hook point is architecture-dependent.
19     For GPT-2, a common stable tap is the block output (transformer.h[1]).
20     """
21    def hook(module, inp, out):
22        # out: [B,T,d]
23        y = out.clone()
24        y[0, t, :] = a_clean_Td[t, :]
25        return y
26    return hook
```

Code Cont

```
1 @torch.no_grad()
2 def patched_score(prompt_crp, target_str, cache_cln, u, t=-1):
3     l, tokpos, kind = u # u=(l,t,"res")
4     assert kind == "res"
5     x = tok(prompt_crp, return_tensors="pt").to(device)
6     tid = tok.encode(target_str, add_special_tokens=False)[0]
7
8     handle = model.transformer.h[1].register_forward_hook(
9         patch_hidden_states_layer(l, tokpos, cache_cln[l])
10    )
11    out = model(**x, output_hidden_states=False)
12    handle.remove()
13    return out.logits[0, t, tid].item()
14
15 # ---- acceptance test ----
16 clean = "Alice gave Bob a book. Bob thanked"
17 corr = "Alice gave Bob a book. Alice thanked"
18 target = " Bob" # tokenization-sensitive
19
20 layers = [4,5,6]
21 cache_cln, _ = cache_residuals(clean, layers)
22
23 base, _ = score_logprob(corr, target)
24 u = (6, -1, "res")
25 patched = patched_score(corr, target, cache_cln, u)
26
27 print("E_u = ", patched - base)
```

Expected outcome

- For some (ℓ, t) , $E_u > 0$ (patch restores target logit).
- Random (ℓ, t) : E_u concentrates near 0 (sanity baseline).

Task 3: Compute the patch-effect tensor $E_u^{(i)}$ (the raw signal)

What will be done

For each example i and node u , compute the patch effect:

$$E_u^{(i)} = O(\tilde{f}_\theta(x_i^{\text{crp}}; u)) - O(f_\theta(x_i^{\text{crp}})).$$

This is the core interventional dataset.

How to do it (engineering)

- Choose an observable O : Δlogit on the correct token is a strong default.
- Store $E_u^{(i)}$ in a sparse-friendly format (chunk by layer/token; compress).
- Add reproducibility: fixed seeds, versioned templates, deterministic batching if possible.

What to expect / acceptance test

- Heatmaps over (ℓ, t) reveal structured “bands” (not uniform noise).
- Effects differ across slices (task family / corruption), enabling classification.

Task 3 (narrowed): Compute a minimal patch-effect tensor $E_{\ell,t}^{(i)}$

Goal (keep scope tight)

- Start with **residual-only** nodes: $u = (\ell, t, \text{res})$.
- Fix a small layer set $\mathcal{L}_0 \subset \{0, \dots, L-1\}$ and token positions $\mathcal{T}_0$.
- Compute the matrix per example:

$$E_{\ell,t}^{(i)} \equiv E_{u=(\ell,t,\text{res})}^{(i)}.$$

Workflow (acceptance test)

- For each pair $(x_i^{\text{cln}}, x_i^{\text{crp}}, y_i^*)$ compute baseline O_i^{base} .
- Cache clean residuals for chosen layers $\ell \in \mathcal{L}_0$.
- For each $(\ell, t) \in \mathcal{L}_0 \times \mathcal{T}_0$, run one patched forward and record:

$$E_{\ell,t}^{(i)} = O_{i,\ell,t}^{\text{patch}} - O_i^{\text{base}}.$$

- Store $E^{(i)}$ as a dense matrix (small) or sparse triplets (large).

What to expect

- For a fixed i , $E_{\ell,t}^{(i)}$ is *structured* (few hot spots).
- Aggregating over i gives stable “important layers/tokens”.

Code skeleton: batch loop + save $E_{\ell,t}^{(i)}$ (residual-only)

```
1 import numpy as np
2 import torch
3 # Assume you already have:
4 # - generate_dataset(N): yields (x_cln, x_crp, y_star, slice_label, meta)
5 # - cache_residuals(prompt, layers): returns cache_cln[layer] = [T,d]
6 # - score_logprob(prompt, target_str): returns O(prompt)
7 # - patched_score(prompt_crp, target_str, cache_cln, u): returns O(patched)
8 L0 = [4, 5, 6]           # chosen layers (tight scope)
9 T0 = [-1]                # chosen token positions (start with last token)
10 N = 200                  # small pilot
11 records = []             # list of dicts per example
12 for i, (x_cln, x_crp, y_star, s, meta) in enumerate(generate_dataset(N, seed=0)):
13     # baseline on corrupted
14     O_base, _ = score_logprob(x_crp, " " + y_star) # tokenization caveat
15     # cache clean activations once per example
16     cache_cln, _ = cache_residuals(x_cln, L0)
17     # compute E matrix for this example
18     E = np.zeros((len(L0), len(T0)), dtype=np.float32)
19     for a, l in enumerate(L0):
20         for b, t in enumerate(T0):
21             u = (l, t, "res")
22             O_patch = patched_score(x_crp, " " + y_star, cache_cln, u, t=t)
23             E[a, b] = O_patch - O_base
24     records.append({
25         "i": i,
26         "slice": s,      # e.g. {"task":"IOI","corruption":"swap"}
27         "meta": meta,    # names, etc.
28         "L0": L0,
29         "T0": T0,
30         "E": E,          # minimal patch-effect tensor
31         "O_base": O_base,
32     })
33 # Save compactly (pilot): numpy npz or parquet later
34 np.savez_compressed("patch_effects_pilot.npz", records=records)
```

Task 4: Build graphs $\{G_s\}$ from effects (standardize + sparsify)

What will be done

Aggregate effects within each slice s into a directed weighted graph

$$G_s = (\mathcal{V}, \mathcal{E}_s, w_s),$$

with **fixed node set** $\mathcal{V}$ and **top- k sparsification**.

How to do it (recommended default)

- Co-influence weights:

$$w_s(u \rightarrow v) = \text{corr} \left(\{E_u^{(i)}\}_{i \in \mathcal{I}_s}, \{E_v^{(i)}\}_{i \in \mathcal{I}_s} \right).$$

- Impose direction constraint $u \prec v$ (by layer/token) to reduce spurious edges.
- Keep top- k outgoing edges per node (same k for all slices).

What to expect / acceptance test

- Graph sizes are comparable across slices (same nodes, similar edge budget).
- Graph statistics (degree, edge-weight histograms) differ meaningfully across task families.

Task 4 (narrowed): Build one sparse graph per slice from $E_{\ell,t}^{(i)}$

Goal (keep scope tight)

- Start with residual-only nodes: $u = (\ell, t, \text{res})$ for $\ell \in \mathcal{L}_0$, $t \in \mathcal{T}_0$.
- Build graphs at the **slice** level: one graph G_s per corruption/task family.
- Use a single edge rule: **co-influence correlation** + top- k sparsification.

Workflow (construction recipe)

- 1 Fix node list $\mathcal{V}_0 = \{(\ell, t, \text{res})\}$ and a direction order $u \prec v$ (e.g. ℓ increasing, then t).
- 2 For each slice s , collect examples $\mathcal{I}_s$ and stack effect vectors $\mathbf{e}^{(i)} \in \mathbb{R}^{|\mathcal{V}_0|}$ where $\mathbf{e}^{(i)}[u] = E_u^{(i)}$.
- 3 Compute pairwise correlations $w_s(u \rightarrow v) = \text{corr}(\mathbf{e}[u], \mathbf{e}[v])$ for $u \prec v$.
- 4 For each node u , keep top- k outgoing edges by $|w_s(u \rightarrow v)|$.
- 5 Store $(\mathcal{V}_0, \mathcal{E}_s, w_s)$ in a lightweight format (edge list).

Acceptance test

- All slices share the same $|\mathcal{V}_0|$ and exactly $\leq k|\mathcal{V}_0|$ edges.
- Edge-weight distribution differs across slices (signal, not identical graphs).

Saved graph format (minimal, kernel-friendly)

- Fixed node list $V0$ (implicit indices $0, \dots, |V0| - 1$).
- Edge list: $(u_idx, v_idx, \text{weight})$.
- Optional: store node attributes (layer ℓ , token t , type).

Task 4 — Implementation: From patch effects to sparse graphs

(A) Node list + slice grouping

```
1 import numpy as np
2
3 def make_node_list(L0, T0):
4     # residual-only nodes
5     return [(1, t, "res") for l in L0 for t in T0]
6
7 def node_order(u):
8     l, t, kind = u
9     return (l, t)
10
11 def build_graph_for_slice(records_s, V0, k=5, directed=True):
12     """
13     records_s: list of per-example dicts with E matrix + (L0,T0)
14     V0: fixed node list (same for all slices)
15     Returns: edge list [(u_idx,v_idx,w), ...]
16     """
17     # Stack effect vectors: shape [n_examples, n_nodes]
18     E_stack = []
19     for rec in records_s:
20         # rec["E"] is [len(L0), len(T0)] matching V0 order
21         e = rec["E"].reshape(-1) # residual-only flatten
22         E_stack.append(e)
23     X = np.asarray(E_stack, dtype=np.float32) # [n, m]
24     n, m = X.shape
25
26     # Precompute correlation matrix (m x m)
27     # (for small m this is fine; later: blockwise / sparse)
28     Xc = X - X.mean(axis=0, keepdims=True)
29     denom = (Xc.std(axis=0, keepdims=True) * 1e-8)
30     Z = Xc / denom
31     C = (Z.T @ Z) / max(n - 1, 1) # approx corr
```

Input assumptions

- Each record contains:
 - E: patch-effect matrix $E_{\ell,t}^{(i)}$
 - slice: task / corruption label
- Node list V_0 is fixed across slices
- Direction constraint: (ℓ, t) order

(B) Build sparse co-influence graph

```
1 # continued build_graph_for_slice
2 # Enforce direction constraint u < v by order
3 idx_order = np.argsort([node_order(u) for u in V0], axis=0)[: ,0] \
4     if isinstance(np.argsort([node_order(u) for u in V0]), np.ndarray) else None
5
6 # Build edges with top-k per u
7 edges = []
8 for ui, u in enumerate(V0):
9     # candidates v with u<v
10    cand = []
11    for vi, v in enumerate(V0):
12        if ui == vi:
13            continue
14        if node_order(u) < node_order(v): # u < v
15            cand.append((vi, C[ui, vi]))
16
17    # top-k by absolute correlation
18    cand.sort(key=lambda z: abs(z[1]), reverse=True)
19    for vi, w in cand[:k]:
20        edges.append((ui, vi, float(w)))
21
22 return edges
23
24 # Example usage:
25 # group records by slice label (task/corruption)
26 def group_by_slice(records):
27     groups = {}
28     for r in records:
29         key = (r["slice"] + ["task"], r["slice"] + ["corruption"])
30         groups.setdefault(key, []).append(r)
31     return groups
32
33 # records loaded from patch_effects_pilot.npz (from Task 3)
34 # groups = group_by_slice(records)
35 # V0 = make_node_list(L0=[4,5,6], T0=[-1])
36 # edges_s = build_graph_for_slice(groups[("IOI", "swap")], V0, k=3)
```

Output (per slice)

- Fixed node indices: $0, \dots, |V_0| - 1$
- Edge list: $(u_idx, v_idx, weight)$
- $\leq k|V_0|$ edges (comparable graphs)

Task 5: Classical baselines (must be strong before quantum)

What will be done

Compute graph embeddings and classical kernels, then run supervised/unsupervised evaluation.

How to do it (baseline stack)

- WL subtree features $\rightarrow$ linear/RBF kernel.
- Spectral vectors (Laplacian eigenvalues) $\rightarrow$ RBF kernel.
- Typed graphlet/motif counts (small motifs) $\rightarrow$ linear/RBF kernel.

What to expect / acceptance test

- You can classify slice labels above chance (task vs corruption).
- Learning curves: accuracy vs #graphs; stability vs graph noise.

Task 5 (narrowed): One strong classical baseline (WL $\rightarrow$ kernel SVM)

Goal (keep scope tight)

- Start with **one** embedding: **WL subtree features** (fast, very strong baseline).
- Start with **one** kernel: linear (then optionally RBF).
- Evaluate on **one label**: slice classification (e.g. corruption type).

Workflow (acceptance test)

- ➊ Convert each slice-graph G_s into a standard graph object (adjacency + node attributes).
- ➋ Compute WL feature vectors $x_s \in \mathbb{R}^d$ for all graphs.
- ➌ Build kernel matrix $K_{st} = x_s^\top x_t$ (linear) or RBF on x .
- ➍ Train SVM (or kernel ridge) and report accuracy/AUC.
- ➎ Plot learning curve: accuracy vs #graphs (subsample S).

What to expect

- Above-chance classification with surprisingly small S (if graphs carry signal).
- WL-depth sweep: depth 1–3 often sufficient for pilot.

Expand only after the pilot works

Spectral embeddings + graphlets are second-tier additions once WL pipeline is stable.

Notes (make it match your graphs)

- Set node attributes: type=res/att/mlp, plus optional layer, token.
- For weighted edges, start by ignoring weights (unweighted WL) in the pilot.
- Later: replace this toy WL with a library WL kernel (GraKel) for robustness.

Task 5 — Implementation: WL features → linear-kernel SVM

(A) WL feature extraction + vectorization

```
1 import numpy as np
2 import networkx as nx
3
4 def wl_features(G, n_iter=2, node_attr="type"):
5     """
6     G: networkx graph with node attributes
7     Returns: dict (label -> count)
8     """
9     labels = {v: str(G.nodes[v].get(node_attr, "0"))
10               for v in G.nodes()}
11     feat = {}
12
13     for it in range(n_iter + 1):
14         # count current labels
15         for v, lab in labels.items():
16             feat[(it, lab)] = feat.get((it, lab), 0) + 1
17
18         # relabel via neighborhood multiset
19         new_labels = {}
20         for v in G.nodes():
21             neigh = sorted(labels[u] for u in G.neighbors(v))
22             new_labels[v] = labels[v] + "|" + "|".join(neigh)
23
24         # hash to keep labels compact
25         labels = {v: str(hash(new_labels[v])) for v in G.nodes()}
26
27     return feat
28
29 def vectorize(feat_dicts):
30     keys = sorted({k for d in feat_dicts for k in d.keys()})
31     X = np.zeros((len(feat_dicts), len(keys)), dtype=np.float32)
32     for i, d in enumerate(feat_dicts):
33         for j, k in enumerate(keys):
34             X[i, j] = d.get(k, 0)
35     return X
```

(B) Train/evaluate a baseline classifier

```
1 from sklearn.svm import SVC
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import accuracy_score
4
5 # graphs: list[nx.Graph], y: list[int/str] slice labels
6 feat_dicts = [wl_features(G, n_iter=2, node_attr="type")
7               for G in graphs]
8 X = vectorize(feat_dicts)
9
10 Xtr, Xte, ytr, yte = train_test_split(
11     X, y, test_size=0.3, random_state=0, stratify=y
12 )
13
14 clf = SVC(kernel="linear")
15 clf.fit(Xtr, ytr)
16
17 pred = clf.predict(Xte)
18 print("acc =", accuracy_score(yte, pred))
```

Minimal tweaks (recommended)

- Set node attribute `type` $\in \{\text{res}, \text{mlp}, \text{att}\}$ (pilot).
- Optional: concatenate layer into the initial label for more structure.
- Sweep `n_iter = 1, 2, 3` and plot accuracy vs #graphs.

Task 6: Quantum kernel “swap” (same embeddings, different geometry)

What will be done

Replace k_{RBF} with a quantum fidelity kernel k_q on the same embedding vectors x_s .

How to do it (minimal viable quantum)

- Choose encoding: angle encoding or data re-uploading; keep qubit count small ($n_q \ll d$ via PCA/random projection).
- Estimate kernel entries with shots M : build $K^{(q)}$.
- Run the same downstream method (SVM/KRR/clustering) and compare against classical baselines.

What to expect / acceptance test

- Kernel alignment and performance vary with circuit depth and M (shot noise).
- You can map regimes where quantum kernels differ: low-sample, noisy graphs, or specific slice families.

Task 6 (narrowed): Quantum kernel “swap” (same embeddings, different geometry)

What will be done

Replace the classical similarity $k(\cdot, \cdot)$ on the same embedding vectors x_s with a quantum fidelity kernel k_q , producing a kernel matrix $K^{(q)}$ used by the *same* downstream pipeline (SVM/KRR/clustering).

Narrowed scope (minimal viable quantum)

- Reuse the Task 5 embeddings $X \in \mathbb{R}^{S \times d}$ (WL features).
- Reduce dimension to n_q (qubits) with PCA or random projection: $x_s \mapsto z_s \in \mathbb{R}^{n_q}$.
- Use a small circuit (angle encoding + shallow entanglement) with depth D .
- Estimate $K_{st}^{(q)} \approx |\langle \phi(z_s) | \phi(z_t) \rangle|^2$ with shots M .

Acceptance test

- The precomputed-kernel SVM runs end-to-end and yields non-trivial accuracy.
- Performance varies sensibly with M (shot noise) and D (expressivity).

Intuition (what changes)

- **Same inputs:** graphs $\rightarrow$ WL features $\rightarrow x_s$ stay fixed.
- **New geometry:** similarity is induced by a quantum feature map $|\phi(z)\rangle = U_\phi(z)|0\rangle$.
- **Kernel entry:** overlap probability of returning to $|0 \dots 0\rangle$ after $U_\phi(z_s)^\dagger U_\phi(z_t)$.
- **Noise knob:** shots M control estimator variance.
- **Expressivity knob:** depth D controls circuit complexity.

Minimum sweeps to report

- $M \in \{200, 500, 2000, 5000\}$ (shot-noise curve)
- $D \in \{1, 2, 3\}$ (depth sweep)
- Compare to classical linear/RBF on the same split

Task 6 — Implementation: PCA → Quantum kernel matrix → Precomputed SVM

(A) Split + PCA to n_q dimensions

```
1 import numpy as np
2 from sklearn.decomposition import PCA
3 from sklearn.svm import SVC
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score
6
7 # Quantum: PennyLane (simulator). Keep nq small.
8 import pennylane as qml
9
10 # X: [S,d] WL feature vectors from Task 5; y: labels
11 Xtr, Xte, ytr, yte, idx_tr, idx_te = train_test_split(
12     X, y, np.arange(len(y)),
13     test_size=0.3, random_state=0, stratify=y
14 )
15
16 nq = 6      # qubits (<< d)
17 depth = 2   # circuit depth
18 M = 2000    # shots
19
20 pca = PCA(n_components=nq, random_state=0)
21 Ztr = pca.fit_transform(Xtr)
22 Zte = pca.transform(Xte)
23 Z = np.zeros((len(X), nq), dtype=np.float32)
24 Z[idx_tr] = Ztr
25 Z[idx_te] = Zte
```

(B) Quantum kernel + precomputed-kernel SVM

```
1 dev = qml.device("default.qubit", wires=nq, shots=M)
2 def feature_map(z, D=2):
3     # angle encoding + simple entangling layers
4     for i in range(nq):
5         qml.RY(z[i], wires=i)
6         for _ in range(D):
7             for i in range(nq-1):
8                 qml.CNOT(wires=[i, i+1])
9             for i in range(nq):
10                 qml.RZ(z[i], wires=i)
11 @qml.qnode(dev)
12 def overlap_prob(z1, z2, D=2):
13     feature_map(z1, D=D)
14     qml.adjoint(feature_map)(z2, D=D)
15     return qml.probs(wires=range(nq))
16 def quantum_kernel(Z, D=2):
17     S = len(Z)
18     K = np.zeros((S, S), dtype=np.float32)
19     for i in range(S):
20         for j in range(S):
21             p = overlap_prob(Z[i], Z[j], D=D)
22             K[i, j] = p[0] # prob of all-zeros = |⟨phi(z1)|phi(z2)⟩|^2
23     return K
24 Kq = quantum_kernel(Z, D=depth)
25 # Train/test on precomputed kernel
26 Ktr = Kq[np.ix_(idx_tr, idx_tr)]
27 Kte = Kq[np.ix_(idx_te, idx_te)]
28 clf = SVC(kernel="precomputed")
29 clf.fit(Ktr, ytr)
30 pred = clf.predict(Kte)
31 print("quantum acc =", accuracy_score(yte, pred))
```


Task 7: Robustness + ablations

What will be done

Quantify how conclusions change under controlled noise and design choices.

How to do it (minimum set)

- Patching noise: add noise to $E_u^{(i)}$ or reduce #examples per slice.
- Graph noise: vary top- k , thresholding, or direction constraints.
- Quantum noise: vary shots M and circuit depth; (optional) simple noise model in simulation.

What to expect / acceptance test (figures)

Accuracy/AUC vs #graphs; accuracy vs noise; kernel spectrum/eigen-decay; shot-noise curves; clustering viz (e.g. kernel PCA).

Task 7 (narrowed): Robustness + ablations (minimum publishable set)

What will be done

Quantify how your conclusions (classification/clustering on $\{G_s\}$) change under controlled perturbations to: (i) patch effects, (ii) graph construction, (iii) quantum estimation.

Narrowed scope (minimum ablations)

- **Patch-noise ablation:** add Gaussian noise to $E_u^{(i)}$ with $\sigma \in \{0, 0.05, 0.1, 0.2\}$ (relative scale).
- **Data ablation:** subsample examples per slice: $n_s \in \{10, 25, 50, 100\}$.
- **Graph ablation:** vary sparsification $k \in \{1, 3, 5, 10\}$ and toggle direction constraint.
- **Quantum ablation:** shots $M \in \{200, 500, 2000, 5000\}$ and depth $D \in \{1, 2, 3\}$.

Acceptance test

Your qualitative claims should be **stable** across reasonable ranges (e.g. WL remains strong; quantum differs in specific regimes, not randomly).

Figures to produce (tight list)

- **Learning curve:** accuracy/AUC vs #graphs (subsample S).
- **Patch noise:** accuracy vs σ (keep everything else fixed).
- **Graph design:** accuracy vs k ; plus “with/without” direction constraint.
- **Quantum shot noise:** accuracy vs M (for each depth D).
- **Kernel spectrum:** eigenvalue decay of K vs $K^{(q)}$ (one plot).
- **Viz:** kernel PCA (2D) of slices for classical vs quantum.

Interpretation (what you're showing)

- Sensitivity to noise $\Rightarrow$ how “real” the signal is.
- Sensitivity to k /constraints $\Rightarrow$ dependence on graph construction.
- Sensitivity to $M, D \Rightarrow$ quantum estimator + expressivity trade-off.

Task 7 — Implementation: Noise/ablation sweeps (classical + quantum)

(A) Patch-noise + data subsampling sweeps

```
1 import numpy as np
2 def add_patch_noise(records, sigma=0.0, seed=0):
3     rng = np.random.default_rng(seed)
4     noisy = []
5     for r in records:
6         r2 = dict(r)
7         E = np.array(r["E"], dtype=np.float32)
8         # noise scale relative to std of E (per example)
9         scale = (E.std() + 1e-8)
10        r2["E"] = E + sigma * scale * rng.standard_normal(E.shape)
11        noisy.append(r2)
12    return noisy
13 def subsample_per_slice(groups, n_per_slice=50, seed=0):
14     rng = np.random.default_rng(seed)
15     out = {}
16     for key, recs in groups.items():
17         if len(recs) <= n_per_slice:
18             out[key] = recs
19         else:
20             idx = rng.choice(len(recs), size=n_per_slice, replace=False)
21             out[key] = [recs[i] for i in idx]
22    return out
23 # Example sweep grid
24 SIGMAS = [0.0, 0.05, 0.1, 0.2]
25 NS      = [10, 25, 50, 100]
26 KS      = [1, 3, 5, 10]      # graph sparsity
27 DIR_ON  = [True, False]     # direction constraint toggle
28 MS      = [200, 500, 2000, 5000] # shots
29 DS      = [1, 2, 3]         # circuit depth
```

(B) Graph + quantum knobs: k , direction, shots M , depth D

```
1 def sweep_all(records, L0, T0):
2     # groups: slice -> list[record]
3     groups = group_by_slice(records)
4     V0 = make_node_list(L0, T0)
5     results = []
6     for sigma in SIGMAS:
7         rec_sigma = add_patch_noise(records, sigma=sigma, seed=0)
8         groups_sigma = group_by_slice(rec_sigma)
9         for n_s in NS:
10            groups_sub = subsample_per_slice(groups_sigma, n_per_slice=n_s, seed=0)
11            for k in KS:
12                # build graphs for each slice
13                graphs, y = [], []
14                for key, recs in groups_sub.items():
15                    edges = build_graph_for_slice(recs, V0, k=k)
16                    G = edges_to_networkx(edges, V0)
17                    graphs.append(G); y.append(key)
18
19                # Classical baseline (WL -> SVM)
20                acc_cl = run_wl_svm(graphs, y)
21                # Quantum swap (uses same embeddings X)
22                for M in MS:
23                    for D in DS:
24                        acc_q = run_quantum_svm(graphs, y, M=M, D=D)
25                        results.append((sigma, n_s, k, M, D, acc_cl, acc_q))
26    return results
```

Outputs

- Store results as a table $\Rightarrow$ make plots: accuracy vs σ , n_s , k , M , D .
- Keep the sweep small at first (few slices, few graphs).

Task 8: Packaging, reproducibility, and “minimum publishable result”

What will be done

Turn the pipeline into a reproducible artifact: code + config + cached datasets + plots.

How to do it

- Version prompt templates, corruption rules, and model hash.
- Cache: (i) patch effects, (ii) graphs, (iii) embeddings, (iv) kernels.
- Provide a single script/notebook to regenerate the key plots.

Minimum publishable result (MPR)

- A new dataset of patch-influence graphs for a controlled task family.
- A rigorous baseline comparison of graph embeddings/kernels.
- A characterization of when quantum kernels behave differently (sample efficiency / robustness / noise sensitivity).

Minimal experiment checklist

Core questions

- Can we classify/cluster interpretability graphs by **task family** or **corruption type**?
- Do kernel choices change **sample efficiency** (accuracy vs #graphs) or **robustness** (noise in patch effects)?
- How do **quantum kernel depth** and **shots** trade off bias/variance?

Figures (minimum set)

Accuracy/AUC vs #graphs; accuracy vs noise; kernel spectrum; clustering visualization (kernel PCA / t-SNE); shot-noise curves for k_q .

Conclusion: What this project establishes

Methodological contribution

- Patching turns mechanistic interpretability into a **dataset of interventions**.
- Aggregating patch effects yields **interpretable, comparable graphs** with a fixed node set.
- Kernel methods provide a **clean abstraction layer** for comparing circuits across tasks.

What success looks like

- Clear structure in patch-effect heatmaps and graph statistics.
- Above-chance classification / meaningful clustering of slices.
- Controlled, interpretable sensitivity to noise, sparsity, and kernel choice.

Empirical takeaways

- Classical graph kernels (WL/spectral) form **strong, necessary baselines**.
- Quantum kernels can be inserted as a **drop-in geometry change** without altering the pipeline.
- Differences between kernels emerge in **specific regimes** (low data, noisy effects, graph sparsity).

Outlook (extensions, not requirements)

- Finer node granularity (heads, MLPs) and path-level patching.
- Alternative graph constructions (direct influence, partial correlations).
- Larger models or cross-model graph comparisons.